

Homework 2: 100 Points

This Homework focusses on Instruction Set Architecture, its principles and attributes.

Please submit your answers in an MS Word document or PDF and upload it in Canvas by the due date. There is a 10% penalty per day for late submission without valid excuse.

Present your work neat, clear, organized, logical, and make your case; don't leave it to our interpretation of your answer.

1. (30 Points) Consider the code sequence:

$C = A + B$

$D = A - E$

$F = C + D$

Assume the values A, B, C, D, E, and F reside in memory. For each Architecture

- I. (10 Points) Accumulator Architecture
- II. (10 Points) Memory- Register Architecture
- III. (10 Points) Register-Register Architecture

write the code assuming the instruction codes (opcode) are 8 bits, memory addresses are 32 bits, and register addresses are 6 bits and CPU has 64 Registers; and create a table which specifies:

- The execution sequence
- The variables that were destroyed in the course of execution
- The overhead instruction just to overcome the loss of data
- Total code size
- The number of bytes of instruction and data moved to or from memory
- The number of overhead data bytes.

- I. (10 Points) Accumulator Architecture

```
1  LOAD A
2  ADD B
3  STORE C
4  LOAD A
5  SUB E
6  STORE D
7  LOAD C
8  ADD D
9  STORE F
```

	<u>Accumulator Architecture</u>
<u>Execution Sequence</u>	9 Instructions
<u>Code Size</u>	45 Bytes (9 instructions x (8-bit opcode + 32 bit memory address))
<u>Bytes Moved to/from Memory</u>	36 Bytes (6 loads (4 each) + 3 stores)

	(4 each))
<u>Variables Destroyed</u>	None
<u>Overhead Instructions</u>	None
<u>Overhead Data Bytes</u>	None

*This is probably wrong. I got confused. Sorry for not attending office hours.

II. (10 Points) Memory- Register Architecture

```

1  LOAD R1, A
2  ADD R1, B
3  STORE R1, C
4  LOAD R2, A
5  SUB R2, E
6  STORE R2, D
7  LOAD R3, C
8  ADD R3, D
9  STORE R3, F

```

	<u>Memory-Register Architecture</u>
<u>Execution Sequence</u>	9 Instructions
<u>Code Size</u>	54 Bytes
<u>Bytes Moved to/from Memory</u>	24 Bytes
<u>Variables Destroyed</u>	None
<u>Overhead Instructions</u>	None
<u>Overhead Data Bytes</u>	None

*This is probably wrong. I got confused. Sorry for not attending office hours.

III. (10 Points) Register-Register Architecture

```

1  LOAD R1, A
2  LOAD R2, B
3  ADD R3, R1, R2
4  LOAD R4, E
5  SUB R5, R1, R4
6  ADD R6, R3, R5
7  STORE R3, C
8  STORE R5, D
9  STORE R6, F

```

	<u>Register-Register Architecture</u>
<u>Execution Sequence</u>	9 Instructions
<u>Code Size</u>	45 Bytes
<u>Bytes Moved to/from Memory</u>	24 Bytes
<u>Variables Destroyed</u>	None
<u>Overhead Instructions</u>	Extra load and store due to
<u>Overhead Data Bytes</u>	None

*This is probably wrong. I got confused. Sorry for not attending office hours.

2. (15 Points) Consider the C++ struct:

```

Struct foo {
    char a;
    bool b;
    int c;
    double d;
    short e;
    float f;
    double g;
    char *cptr;
    float *fptr;
    int x;
};

```

1. (5 Points) What is the size of foo struct?

56 Bytes Total (48 bytes of data and 8 bytes to be used as padding for the 8-byte alignment)

2. (5 Points) What is the memory size required?

48 bytes for memory is required.

3. (5 Points) What is the minimum memory size required if you re-arrange the struct?

Assume the following data type and sizes for the above struct in a 64-bit machine.

<u>Data Type</u>	<u>Data size on 64-bit machine (bytes)</u>
Char	1
Bool	1
Int	4
Long	8
Double	8
Short	2
Float	4
pointer	8

48 bytes is required for memory (for re-arranged).

3. (10 Points) Assume the following instruction mix for a RISC-V like RISC Program:

- 15% of instructions are store
- 25% of instructions are load
- 15% of instructions are branches
- 35% of instructions are integer arithmetic /logical
- 5% of instructions are floating points
- The rest of instructions (shift, move, etc.) are like arithmetic/logical instructions.

Further assume that:

- Load and store instructions take 4 cycles each, branches take 3 cycles, arithmetic/logical instruction (and their likes) each takes one cycle, and floating point instructions take 12 cycles.

Compute the overall CPI of this RISC Program.

Store: $0.15 * 4 = 0.60$

Load: $0.25 * 4 = 1.00$

Branch: $0.15 * 3 = 0.45$

Arithmetic/Logical: $(0.35 + 0.05) * 1 = 0.40$

Floating Point: $0.05 * 12 + 0.60$

$0.60 + 1 + 0.45 + 0.40 + 0.60 = 3.05$

CPI: 3.05

4. (20 Points) Write the following code in RISC-V. Assume A, B, C, and D are DP (Double–Precision) floating points, I and J are integers. Include comments on each line.

$C = A - B$

$D = 2 - A + B$

If $I = J$ then $A = A + B$; else $B = B - A$

```
1  # Load A, B, C, D into floating-point registers (f0, f1, f2, f3)
2  fld f0, 0(xA)      # Load A into f0
3  fld f1, 0(xB)      # Load B into f1
4
5  # C = A - B
6  fsub.d f2, f0, f1   # f2 = f0 - f1 (C = A - B)
7  fsd f2, 0(xC)      # Store C back to memory
8
9  # D = 2 - A + B
10 li t0, 2           # Load 2 into t0
11 fcvt.d.w f4, t0     # Convert integer 2 to double and store in f4
12 fsub.d f3, f4, f0   # f3 = 2 - A
13 fadd.d f3, f3, f1   # f3 = (2 - A) + B
14 fsd f3, 0(xD)      # Store D back to memory
15
16 # if (I == J)
17 lw t1, 0(xI)        # Load I into t1
18 lw t2, 0(xJ)        # Load J into t2
19 beq t1, t2, if_true # If I == J, jump to if_true
20
21 # else: B = B - A
22 fsub.d f1, f1, f0   # f1 = B - A
23 fsd f1, 0(xB)      # Store updated B back to memory
24 j end_if           # Skip the if_true block
25
26 if_true:
27 # A = A + B
28 fadd.d f0, f0, f1   # f0 = A + B
29 fsd f0, 0(xA)      # Store updated A back to memory
30
```

5. (25 Points) Consider the code segment

for ($i=0$; $i \leq 100$; $i++$)

$X[i] = X[i] + d$

Assume X is an array of 64-bit integer, d is a 64-bit integer, and i is a 32-bit integer.

Assume $X[0]$ is at address $0(x0)$ in memory, $X[1]$ at $x0 + 8$, $X[2]$ at $x0 + 16$, etc.

1. (15 Points) Write the code in RISC-V; include comments on each line.

```
# Load the base address of X, the constant d, and initialize i
la t0, X          # Load base address of X into t0
li t1, 0          # Initialize i = 0 in t1
li t2, 100        # Set the loop limit (100) in t2
ld t3, 0(xd)      # Load d into t3 (64-bit integer)

loop:
    slli t4, t1, 3    # Multiply i by 8 (shift left by 3) to get the memory offset
    add t5, t0, t4    # Compute the address of X[i]
    ld t6, 0(t5)      # Load X[i] into t6
    add t6, t6, t3     # Add d to X[i]
    sd t6, 0(t5)      # Store the updated value back into X[i]
    addi t1, t1, 1     # Increment i
    ble t1, t2, loop  # Check if i <= 100; if true, branch to loop
```

2. (5 Points) What is the size of the RISC-V code in bytes?

13 Instructions * 4 bytes= 52 bytes

3. (5 Points) What is the RISC-V code CPI? Assume Load and store instructions take 4 cycles each, branches take 3 cycles, and arithmetic/logical instruction each takes one cycle.

Load and Store: $2 * 4 = 8$ cycles per iteration

Arithmetic/Logical Instructions: $3 * 1 = 3$ cycles per iteration

Branch Instruction: $1 * 3 = 3$ cycles per iteration

$8 + 3 + 3 = 14$ cycles

14 cycles * 101 iterations from RISC-V code = 1414 instructions

CPI= $1414 / 1010 = 1.4$